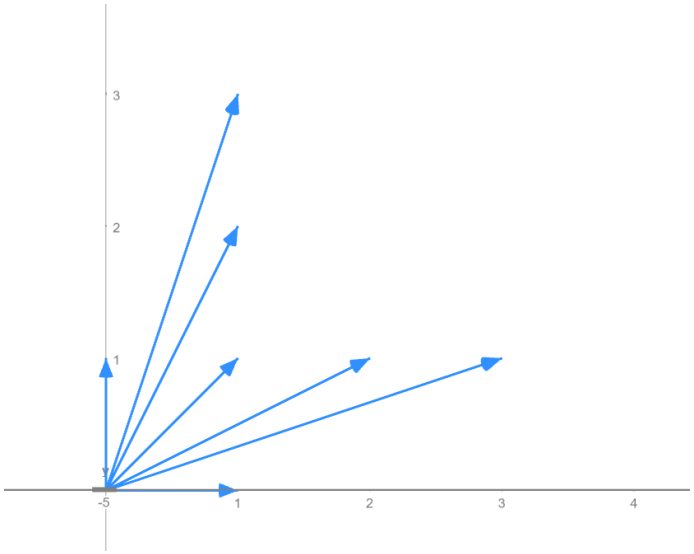
Process to Use List of Set Vectors:

1. **Memory Inefficiency.**
    a. In a 2D environment, there are only 7 vectors we are really working from. Seen below



    b. However, in a 3D environment the number of vectors jumps very sharply

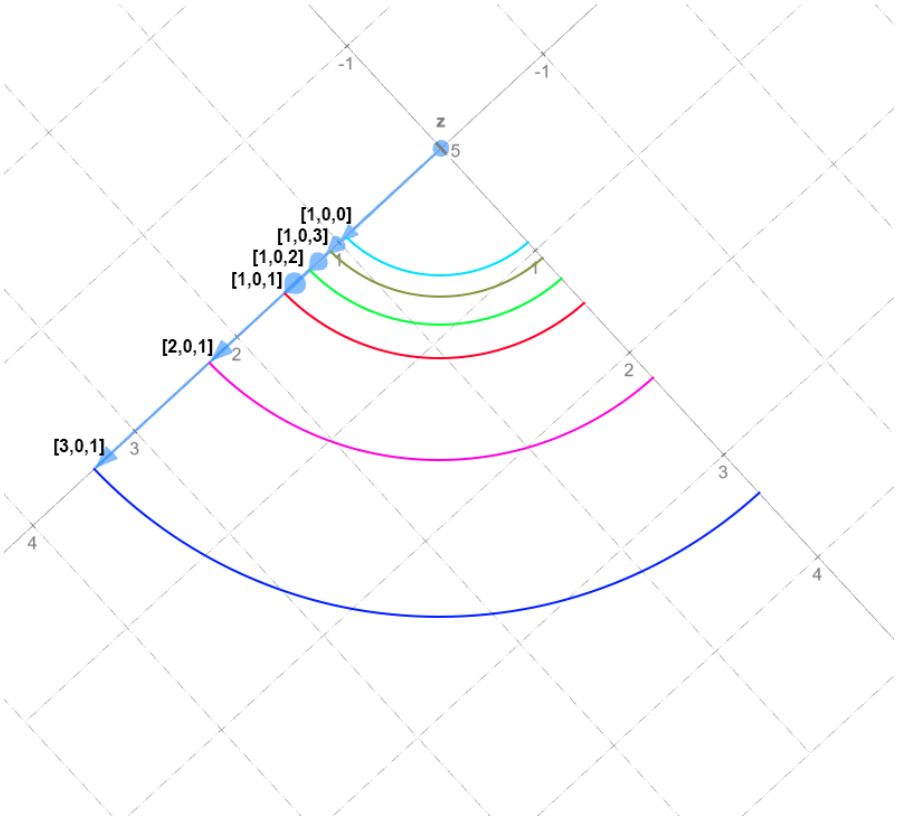


Figure 1 Sweeping XZ vectors around the Z axis

c. To create these 3D vectors we would need to make large set of vectors that may not accurately encompass the space.

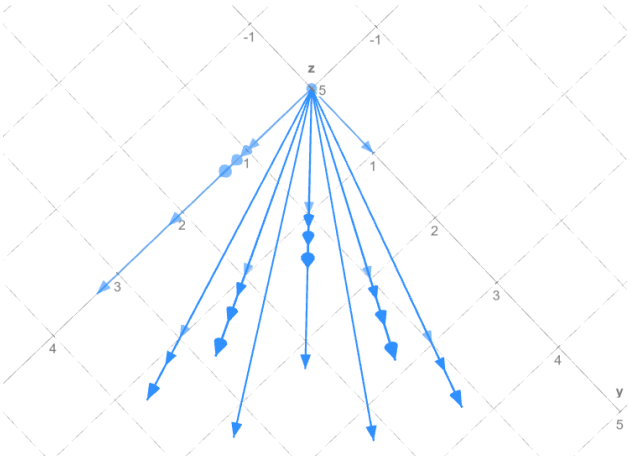


Figure 2 3D vectors top-down view

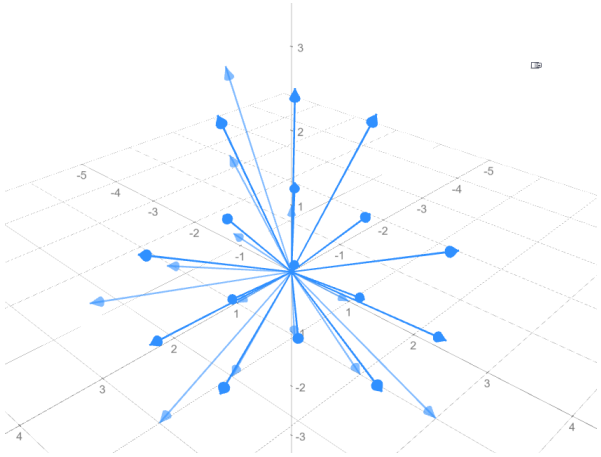


Figure 2 3D vectors direct view

d. That is to say, the length of the set of vectors to save is ~36+ (trimmed down to remove vectors that are scalar multiples).
e. But we only ever need some mixture of 4 of them at any puzzle.
f. An 88% memory waste
g. Additionally, every obstacle generates its own answers and goals. So multiply the memory waste by every question/obstacle in the level.

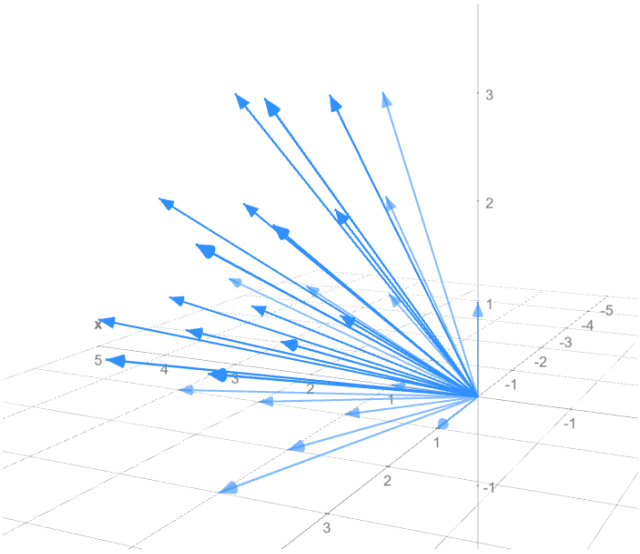


Figure 4 3D vectors side view

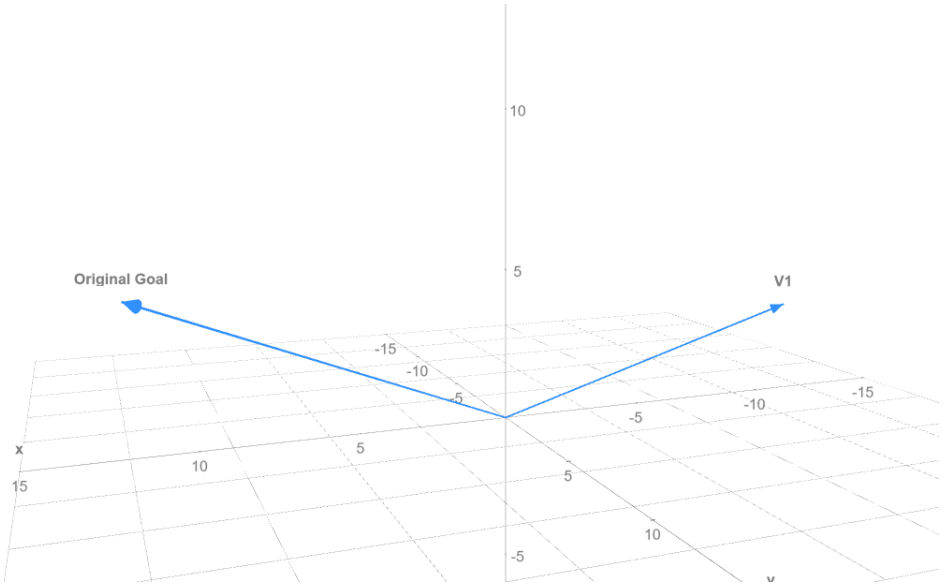
2. Memory Solution

a. Rather than reserve memory unnecessarily holding a set of reference vectors, we can generate them dynamically
b. Consider: Every vector (in this example) has a max magnitude of 3 in any one dimension.
c. We can represent the set of the vectors as:

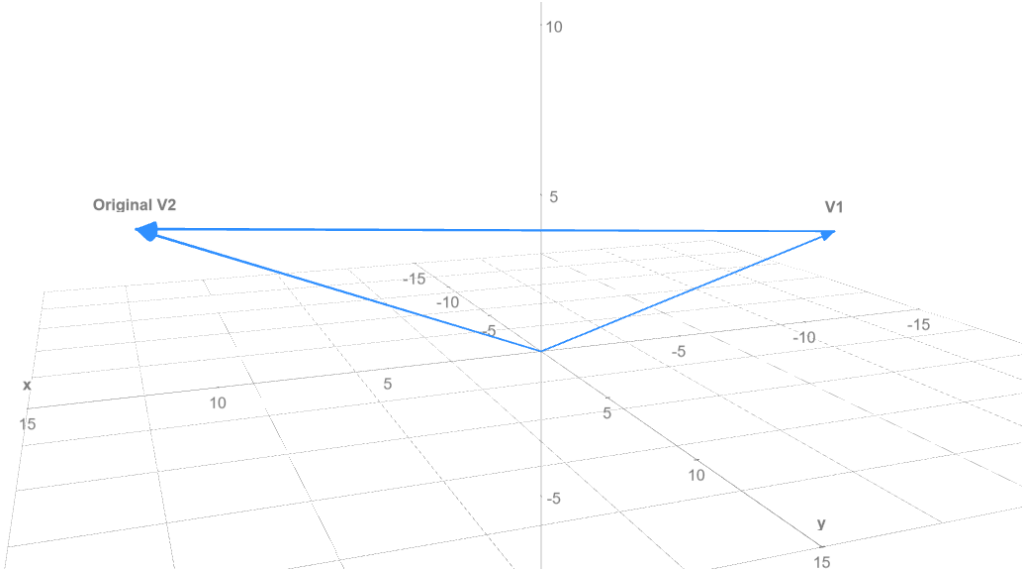
$$V = \{ [x,y,z] \mid x,y,z \geq 0 \text{ and } x,y,z \leq 3 \text{ and } x=y=z \neq 0 \}$$

d. In this way, we can still generate any vector without creating waste.
e. To represent other quadrants we can “flip a coin” for each axis to multiply by -1
f. Downside:
    i. we may end up with some scalar multiples with this approach.

3. **Secondary Vector Problems**
    a. In the current iteration, generating the pair vector works like this:

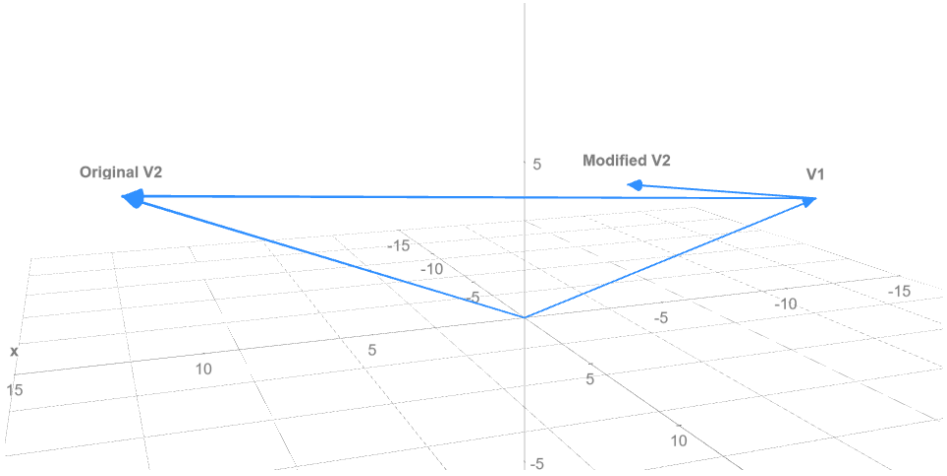


    i. Find the displacement from the Goal [12,8,6] to Vector1 ([-2,1,1] * 4) (ex: [12,8,6] – ([-2,1,1] * 4) = [20,4,2])

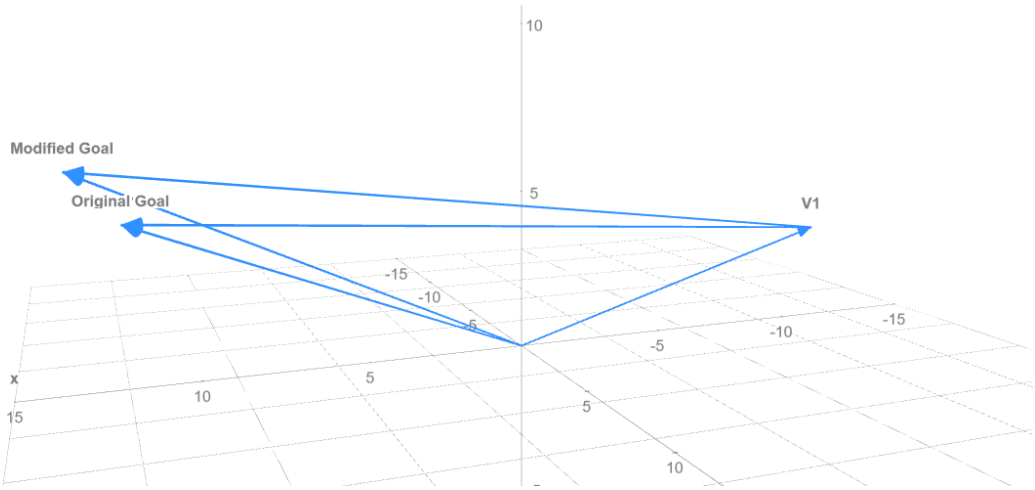


    ii. Divide that vector by some random scalar (ex: [20,4,2] * (1/3) = [6.66,1.33,0.66])

iii. Round off the dimensions (ex [7,1,1])

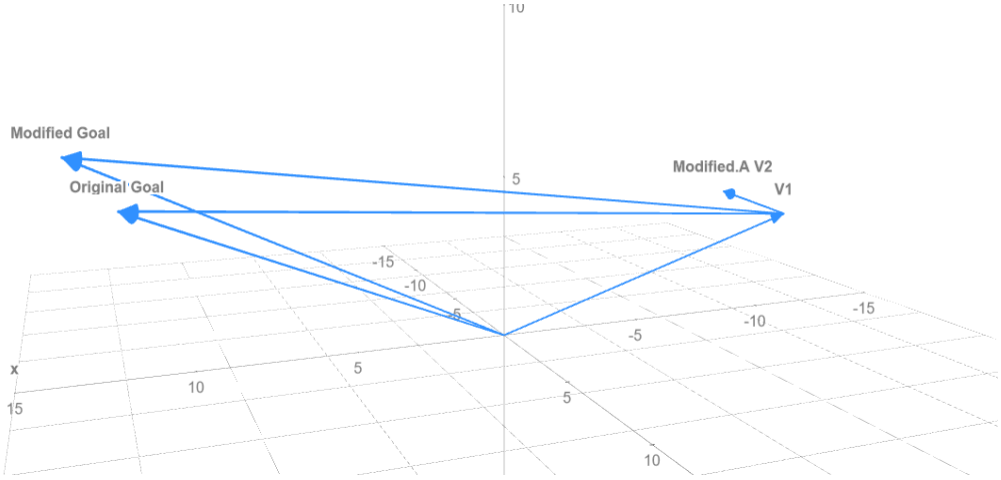


iv. Change the goal to match new pair ([-2,1,1] * 4) + ([7,1,1] * 3) = [13,7,7])

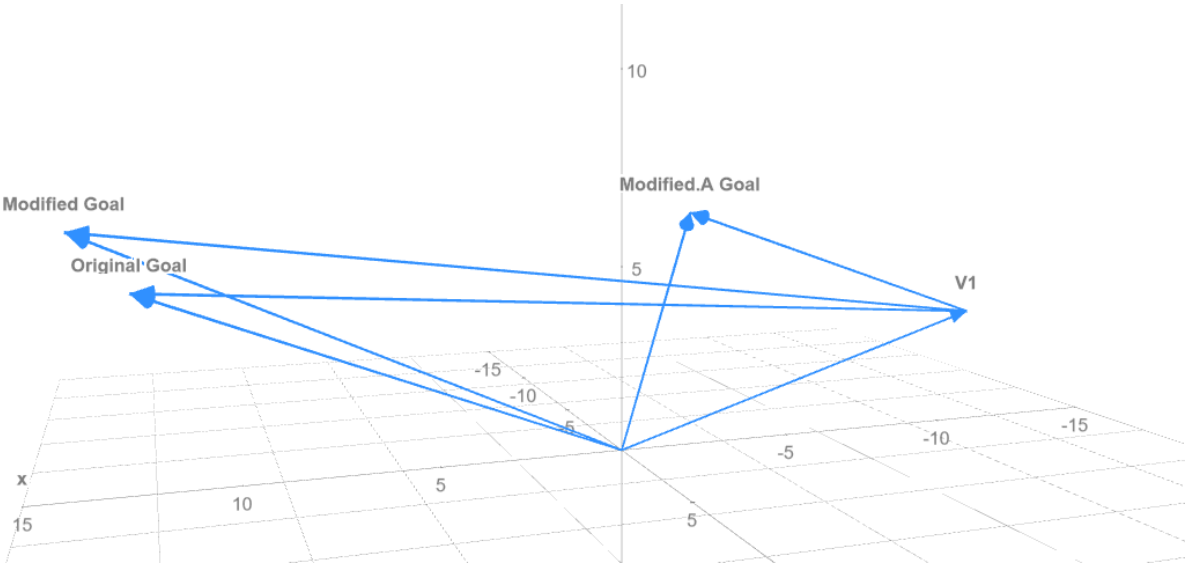


b. Downsides
   i. This presents a problem, though, because [7,1,1] is not in V.
   ii. This means that when the player is presented with possible answer cards, 3 cards will be visually close to each other (in the set of V). But one card may be an obvious oddball. And that oddball will always be apart of the correct answer
   iii. Additionally, this system is liable to create very awkward vectors that are exactly the problem we're trying to fix.
   iv. This may skew results as players will not think critically about what the vectors directions are, but rather that the game has provided half the answer for them.

4. **Possible Secondary Vector Solutions A:**
    a. Simplest solution, forcing the second vector to be in V
    b. Rescale any dimension greater than 3 to equal 3 (ex [7,1,1] becomes [3,1,1]



    c. Keep original scalar and change goal to new pair
    ([-2,1,1] * 4) + ([3,1,1] * 3) = [1,7,7])



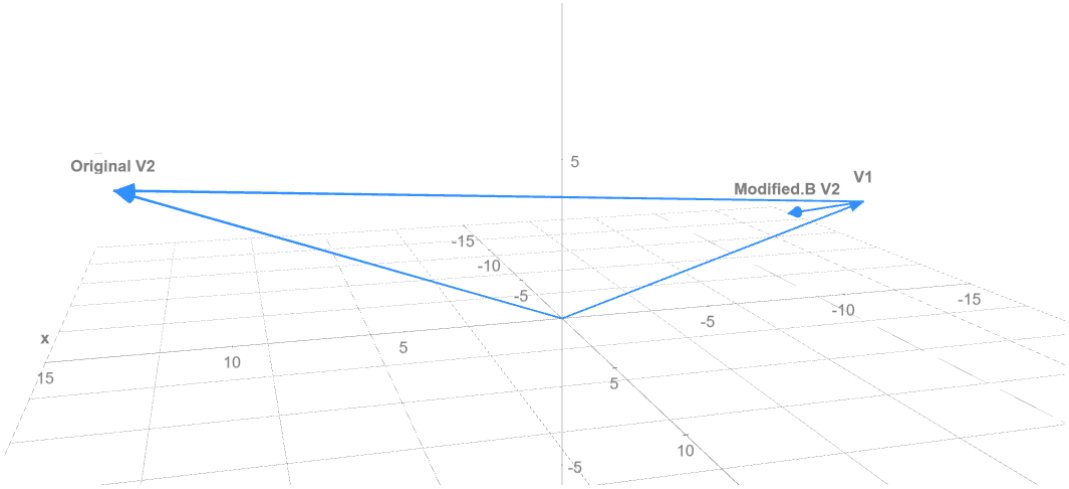
    d. Downsides:
        i. newly generated Vector2 and modified goal are nowhere close to the original set by puzzle.

5. **Possible Secondary Vector Solutions B:**
    a. Find a scalar that can divide the original V2 where no dimension is greater than 3
        i. Original V2 = [20,4,2]
        ii. Choose the largest dimension, x = 20

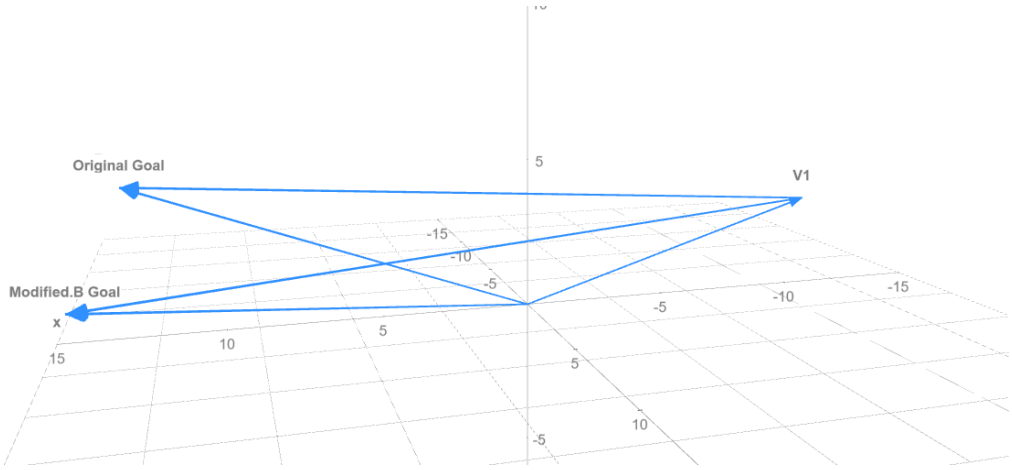
iii. $20/s \leq 3$
iv. $s = 7$

b. Same as before, we'll divide our original V2 by this new scalar
(ex [20,4,2] * (1/7) = [2.86, 0.57, 0.29])

c. Round the results
[3, 1, 0]

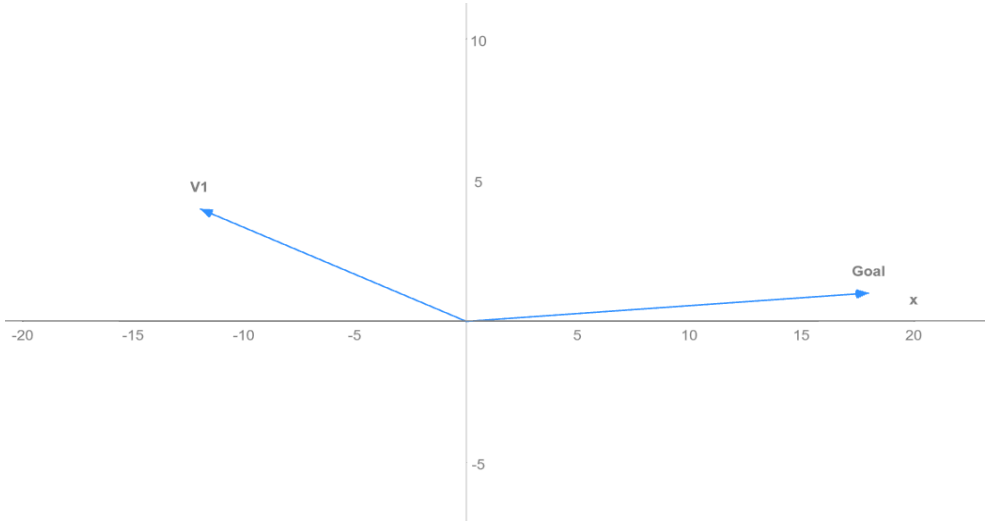


d. Change Goal to match new vector pair:
([-2,1,1] * 4) + ([3,1,0] * 7) = [13,11,4])

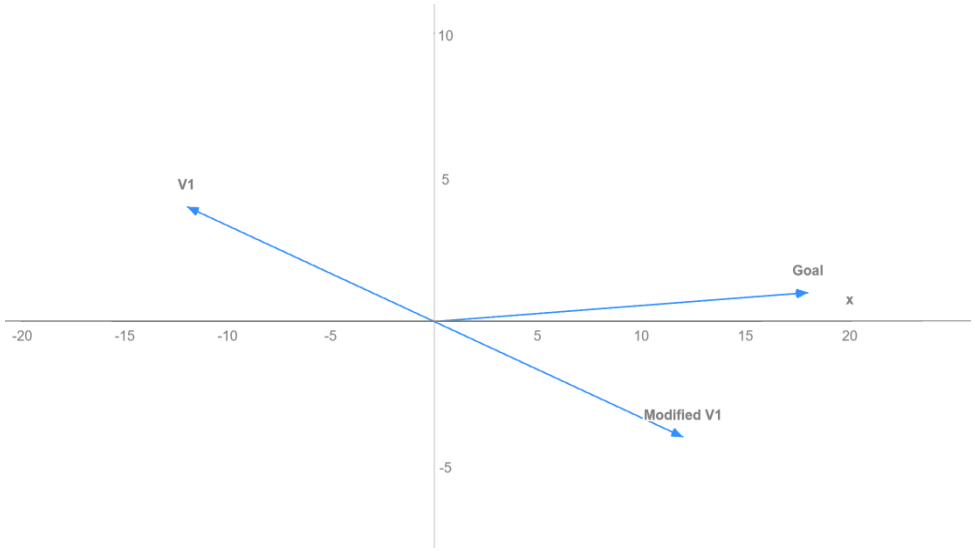


e. Downsides:
   i. This approach takes a bit more calculation time (not much but multiplied by each obstacle in the level).
   ii. The major problem is that "s" may end up being a number larger than 10, which is the maximum size of the game's sliders.
      1. On top of it not being answerable for the players, this may cause an infinite loop as the system locks trying to attempt solutions that can't be used
      2. This can be curbed somewhat. On the rare occasions the system requires "s" to be larger than 10 to make $V2 \in V$, we instead reject it and choose "s" manually as some number between 10 and 5.

6. **Other Background Tricks to Keeping Card Values within V**
    a. Before calculating V2, ensure that we're calculating the displacement from Goal to V1 from the smallest magnitude.
    b. The system may choose V1 to be in the opposite direction of the Goal or even parallel and opposite.
    (ex: Goal = [18,0,1], V1 = [-3,0,1] * 4)



    c. If we let the system create V2 from these two, we can already see that the displacement for x will be >30. Meaning, even if V2 picked 3 for x, s would have to be > 10.
    d. However, the player has the option to slide scalars into both +10 and -10
    e. Knowing this, we can modify V1 without changing its original composition.
    f. The system can check if multiplying V1's scalar by -1 results in a shorter displacement magnitude.



g. Because the player can choose both 4 and -4 when creating their answers, we can leave V1 essentially untouched.
h. However, when we generate V2, we will generate it based off the Modified V1.
i. This helps lessen the instances where “s” may exceed the maximum of 10
j. Downsides:
    i. This creates a system where the correct pair of answer vectors will always be towards-ish the goal
    ii. This does not solve for the rare cases of an orthogonal V1.
        1. In an extremely rare case where V1 is both orthogonal AND the “s” required exceeds 10, we will revert to manually choosing “s” and adjusting the goal to match.